

A firm that remembers, works, and learns

The systems we designed today

memoryNet

Find the work and its evidence

Persona manager

Define the job and its skills

Runtime

Run and coordinate the agents

Evolution engine

Test better ways of working

Three goals

Survive interruptions

Keep learning evidence

Improve without chaos

Four systems, four different responsibilities

Memory and role definitions feed each run; every run should leave evidence for later review.

01 memoryNet

Threads, identities, original evidence, outputs, judgments

LOCAL SCAFFOLD

02 Persona manager

Roles, OODA loops, skills, standards, escalation paths

STARTERS + DRAFTS

03 Runtime

Start agents, assign tasks, deliver messages, collect results

PLANNED

04 Evolution engine

Compare role versions under controlled historical tests

PLANNED

The runtime uses memory and personas. The evolution engine tests changes before they are adopted.

memoryNet keeps four kinds of evidence

A handoff helps the next agent move quickly. The original trace lets it check what actually happened.

Core trace

Recorded messages, actions, tool inputs and results, errors, and ordering.

Investigation and training evidence

Checkpoint

Current state, unfinished work, open questions, and the next action.

Fast continuation

Synthesis

Conclusions, explanations, and lessons drawn from the work.

Retrieval and understanding

Assessment

What succeeded or failed, the supporting evidence, and who judged it.

Evaluation and learning

A saved pointer or Rose summary is not proof that the raw transcript is backed up.

Identity follows the work without inventing agents

A thread can span many sessions. One session can contain several separately identified agents.

THREAD / the continuing line of work

Session A

Provider + native session ID

Session B

Continues the same thread

Agent A1

Agent A2

Keep the identifiers separate

Native IDs

Original session and agent identities

Catalogue UUID

The indexed record in memoryNet

Persona / skill ID

Reusable definition, with version

Fingerprint

Checks exact content; not quality

Execution parent $\neq$ manager $\neq$ reviewer

Unknown native IDs stay unknown. New agents continue the thread with their own identities.

A new agent needs two inputs

Choose an agent type and a thread ID; the same API feeds the human inspector and the agent.

The request

Agent type
data_engineer

Thread ID
The saved thread
UUID



The resume response

Selected persona + skill text
Continuation brief
Original source references
Outputs, capture dates, and
gaps



New execution

Read the relevant
evidence
Take the current
assignment
Work or escalate
Leave a linked
checkpoint

```
GET /api/thread/resume?id=<thread UUID>&role=data_engineer&format=json
```

Every agent follows the same small loop

Its persona defines the job, standards, relationships, and when to ask for help.

OBSERVE

Read the assignment, prior work, and current evidence.



ORIENT

Resolve definitions, uncertainty, dependencies, and scope.

ACT + CHECK

Produce the output, check it, and record the result.



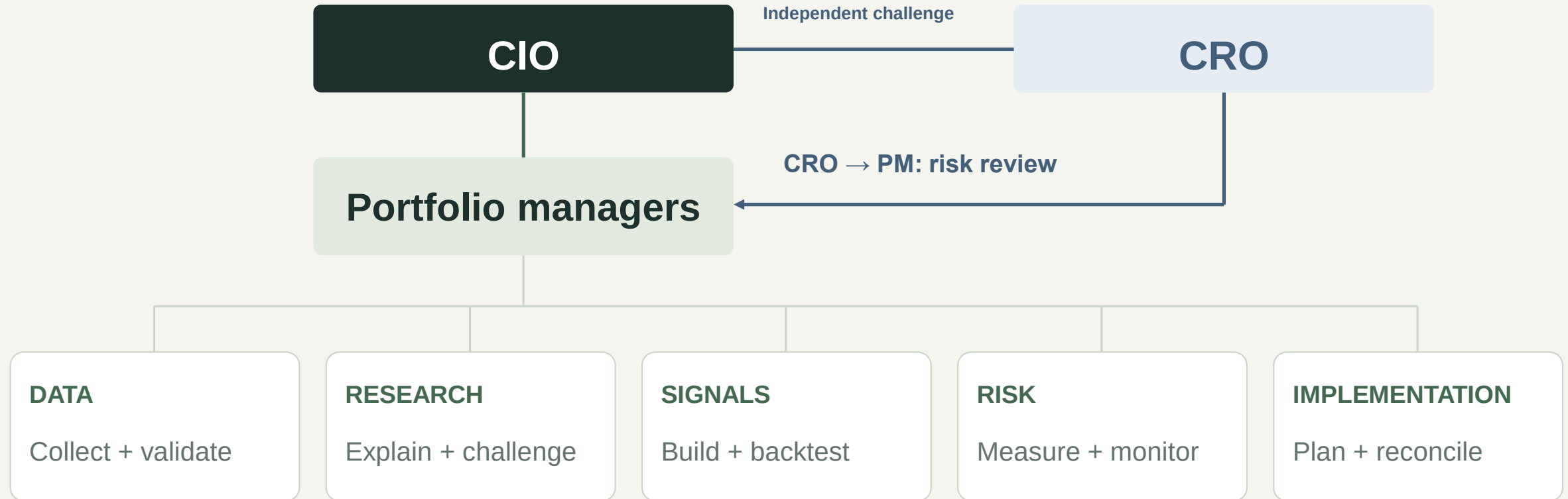
DECIDE

Choose the next action, a targeted question, or an escalation.

Escalation names the gap, needed input, owner, and deadline. A tag alone does not deliver a message.

The firm coordinates work and independent judgment

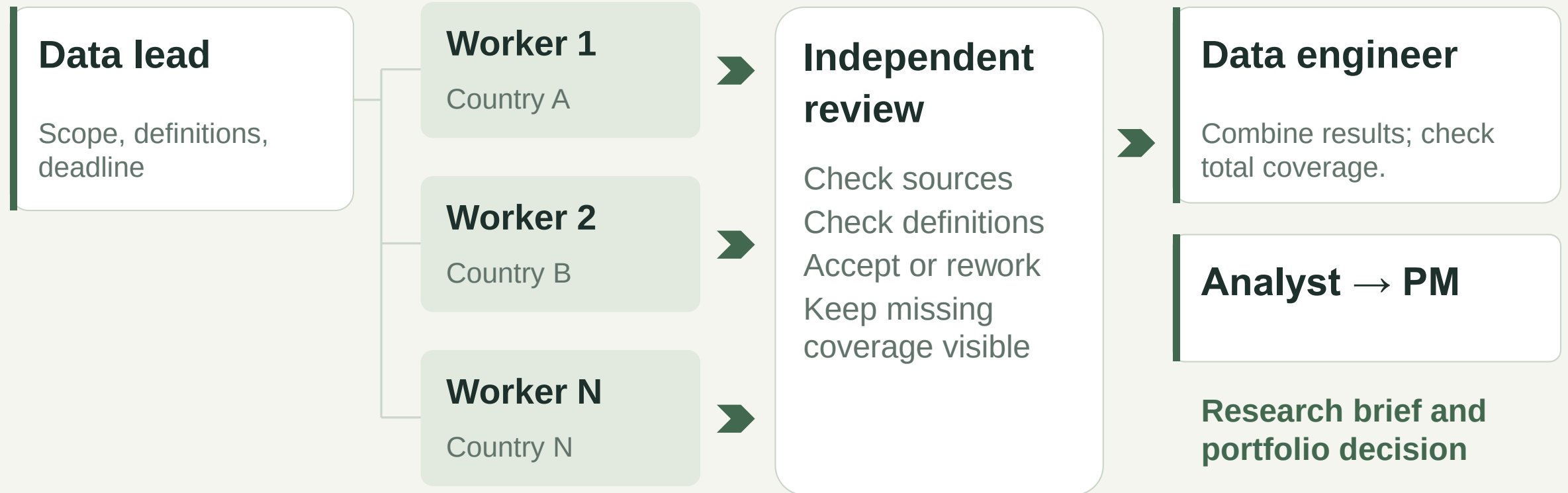
PMs own portfolio decisions. CIO coordinates investment priorities; CRO supplies an independent risk view.



Firm loop: Frame → Observe → Assign → Produce + challenge → Decide → Act → Review + learn

Many workers can share one persona

Example: collect oil-demand evidence country by country, then review it before synthesis.



One researcher persona; many task scopes and execution IDs. Every returned result still needs a judgment.

Skills are procedures for finishing real jobs

The persona owns responsibilities. A skill supplies the inputs, steps, checks, and expected output.

Development	Change a repo • fix a bug • edit a website • deploy and roll back
Data engineering	Pull vendor data • align units and dates • publish and refresh
Research + quant	Write a brief • build a calculation • verify and backtest a signal
Risk + implementation	Reconcile records • calculate scenarios • check expiries • plan execution
PM / CIO / CRO	Assign work • review evidence • compare proposals • record decisions
Client service	Read client context • explain approved findings • prepare reports

35 library entries = 23 procedure drafts + 6 starter files + 6 named placeholders

Rose and vendor access need explicit procedures

Use a shared data workflow, with the connection details for the particular vendor loaded when needed.

Seven Rose jobs

- 1 Read an object
- 2 Follow a map
- 3 Choose a code
- 4 Upload a table
- 5 Build a calculated series
- 6 Refresh a dataset
- 7 Make a notebook

One recipe per provider

Client / host + access

Identifiers + definitions

Dates + expected response

Haver: local recipe documented

Grove BBG updater: calls unfinished

Yahoo refresh: endpoint to resolve

Naming starts with the project's existing convention, then an exact collision check. No invented global pattern.

The risk work reveals three different historical jobs

Each answers a different question and needs different evidence.

Broker reconciliation

What actually happened?

Match snapshots to activity statements and contract records.

Output: explained changes and unresolved differences.

Historical book replay

What if this book stayed fixed?

Reprice a dated set of positions through market history.

Output: a hypothetical P&L path and assumptions.

Signal backtest

What would this rule have done?

Change positions using the information available at each date.

Output: trades, returns, costs, and sensitivity.

Alongside these: current exposures, scenario shocks, expiry review, and data-freshness monitoring.

Learn from old work before writing more instructions

A cheaper model finds candidate lessons; Campbell and a reviewer decide what belongs in the system.

SCREEN

Corrections,
frustration, praise,
failures, successes,
neutral examples

READ

Read the
surrounding episode
and inspect the
outcome evidence

PROPOSE

A scoped principle
with source quotes
and uncertainty

SORT

Persona, skill,
process fix, more
evidence, or dismiss

The inbox records human review. It does not rewrite active instructions.

A reaction is a lead, not proof. A principle becomes a skill only when it becomes a concrete procedure.

Evolution means controlled comparisons

Change one role specification while keeping the other agents and test conditions fixed.

Keep fixed

Other role versions
Model and tools
Data known at each date
Tasks and budgets
Scoring rules

Baseline role

Same historical tasks.
Save the evidence.



Candidate role

Same historical tasks.
Save the evidence.



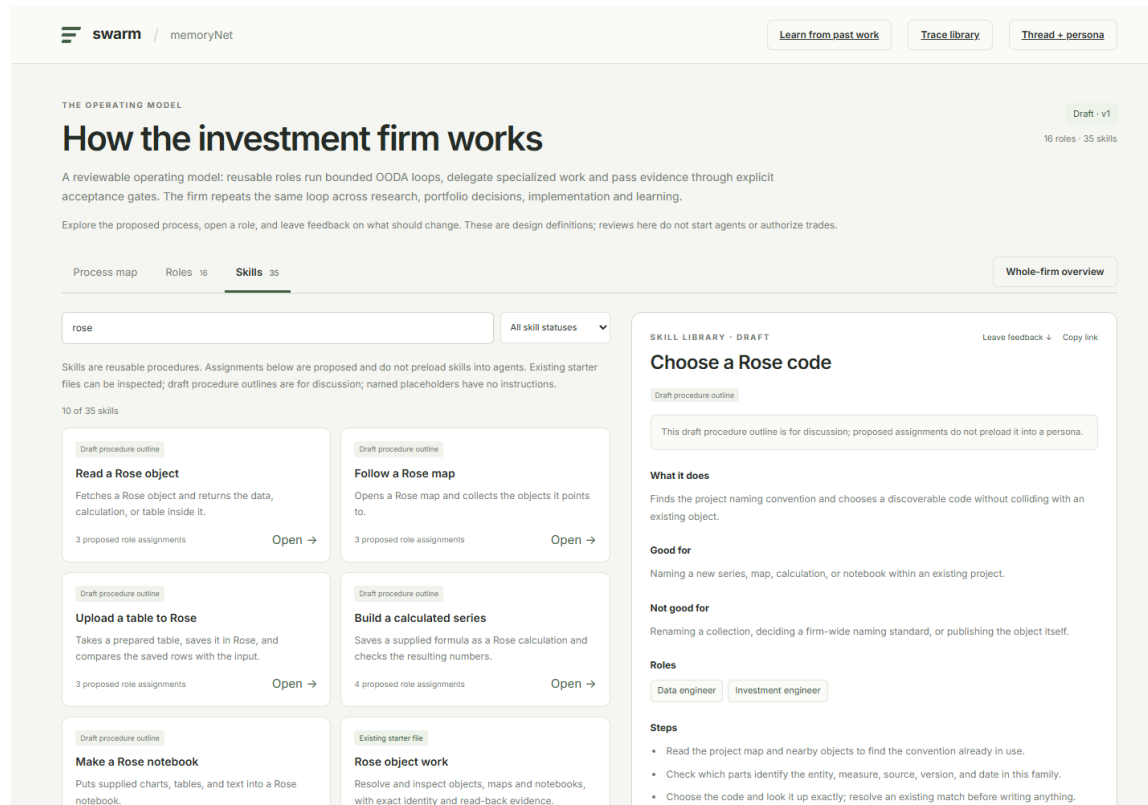
Compare and review

Task quality and correctness
P&L, risk, and exposures
Time, tokens, and cost
Human judgment
Stability across test periods

Tune on one set of periods; evaluate on held-out periods. Adopt a new version only after review.

The local interface makes the design inspectable

People can see the same saved context and role definitions that another agent can retrieve.



Trace library

Find records and inspect source events.

Thread + persona

Load role, skills, saved work, and gaps.

Firm + skills

Click through roles, procedures, and feedback.

Learning inbox

Read evidence and sort proposed lessons.

What exists today—and what still needs building

The design is broader than the running software. Keep that boundary visible.

WORKING LOCAL TOOLS

Catalogue, keyword search,
and source-event lookup

Threads and resume API

Versioned persona + starter-
skill loading

Firm, skill, and learning review
screens

DEFINED FOR REVIEW

16-role firm and
communication map

Agent and firm OODA loops

23 draft task procedures

Escalation and independent
review rules

NEXT ENGINEERING

Reliable capture and off-
machine backup

Automatic run registration and
dispatch

Bound message and
escalation delivery

Historical role evaluation

Local model search depends on its configured model server. A local catalogue is not a remote archive.

Next: prove one complete loop

Start with one thread, one role, one useful output, and one independent judgment.

1

Resume

Supply the role,
thread ID, and
current assignment.

2

Produce

Read the evidence.
Do the task.
Return the result.

3

Review

Accept it, request
corrections, or
identify a concrete
blocker.

4

Preserve

Link the trace,
checkpoint, output,
and judgment to the
thread.

Automate the loop. Compare role versions. Preserve the evidence.